

Dr. B.R. Ambedkar National Institute of Technology Jalandhar

Department of Computer Science and Engineering

UNIT 2

Waterfall Testing Review

Chapters 6 - 11 | Static and Dynamic Testing across the Life Cycle

Software Quality Assurance and Automation Testing

B.Tech Final Year | VII Semester | 2025-26

Source: Software Testing and Continuous Quality Improvement, 3rd Ed., W. E. Lewis, Taylor & Francis, 2009 (pp. 85-136)

Unit Roadmap

6

Overview

Waterfall methodology, PDCA phased approach, the software test plan, technical reviews and inspections

7

Static Testing the Requirements

Ambiguity reviews, requirements defect recording, traceability and the requirements/test matrix

8

Static Testing the Logical Design

Data model, process model, CRUD matrix, logical design defect recording

9

Static Testing the Physical Design

Design representation schemes, coupling, integration test case methodology

10

Static Testing the Program Unit Design

Sequence, selection and iteration constructs; creating unit test cases

11

Static and Dynamic Testing the Code

Programming standards, executing the tests, unit to integration to system to acceptance

Unit Learning Objectives

1

Life-cycle testing as a parallel activity

Testing occurs in parallel with the development life cycle and is a continuous process, not a terminal phase after coding.

2

Deming process improvement applied

The PDCA quality circle, principles and statistical techniques are applied directly to software testing processes.

3

Psychology of life-cycle development and testing

Why testing is more effective when performed outside the development organisation by an independent party.

4

Components of a good test

The test plan is the bible of software testing - its objectives, scope, strategy, approach and test details.

5

Technical review and testing as verification techniques

The two major quality assurance verification approaches for each life-cycle phase.

CH 6

Overview

Pages 87 - 106

The waterfall life-cycle development methodology and the associated testing activities. Deming continuous quality improvement is applied with technical review and testing techniques.

Waterfall Development Methodology

Chapter 6 - Figure 6.1

The life-cycle or waterfall approach breaks the development cycle down into discrete phases, each with a rigid sequential beginning and end. Each phase is fully completed before the next is started. Once a phase is completed, in theory during development, one never goes back to change it.

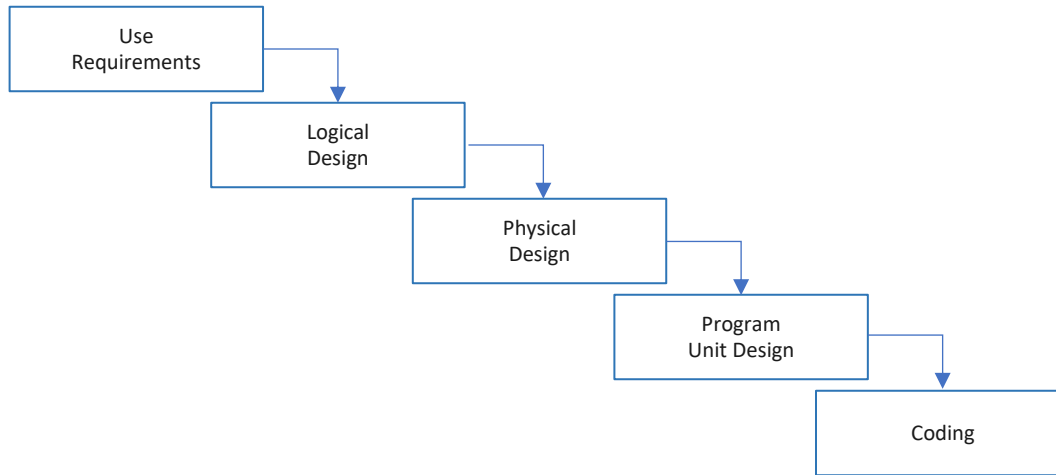


Figure 6.1 - Waterfall development methodology.

Note: the operation and maintenance phase is not shown in Figure 6.1. Defects introduced during the life-cycle phases are detected and corrected there, and new enhancements are incorporated into the application.

Continuous Improvement: The Phased Approach

Deming continuous improvement process is effectively applied to the waterfall development cycle using the Deming quality cycle, or PDCA - that is, Plan, Do, Check and Act. It is applied from two points of view.

Point of View 1 - Software Testing

Verifies and validates that the software design, code and documentation meet all the requirements imposed on them.

Focus: test planning, test design, test development, and test execution.

Point of View 2 - Quality Control / Technical Reviews

The process and methods used to monitor work and observe whether requirements are met.

Focus: structured walkthroughs and inspections to remove defects introduced during the SDLC.

The three major components of quality assurance (Section 1, Software Quality in Perspective)

Software Testing

Quality Control

Software Configuration Management

Psychology of Life-Cycle Testing

In the waterfall life cycle there is typically a concerted effort to keep the testing and development departments separate, with a different reporting structure. Because requirements and design documents have been created at specific phases, a separate quality assurance organisation should be able to translate these documents into test plans, test cases and test specifications.

Two underlying assumptions

1. Programmers should not test their own programs.

2. Programming organisations should not test their own programs.

The reasoning behind each

- Software testing is a destructive process; it is very difficult for a programmer to suddenly change perspective from developing a product to trying to break it.
- It is believed that programmers cannot effectively test their own programs because they cannot bring themselves to attempt to expose errors.
- Errors arise from the programmer misunderstanding of the requirements - a programmer testing his own code would have the same bias.
- A programming organisation is measured by its ability to produce a system on time and economically. If a concerted effort were made to find as many defects as possible, the project would probably be late and not cost effective. Less quality is the result.
- From a practical point of view, an independent organisation should be responsible for the quality of the software products.

Software Testing as a Continuous Improvement Process

Chapter 6 - Figure 6.2

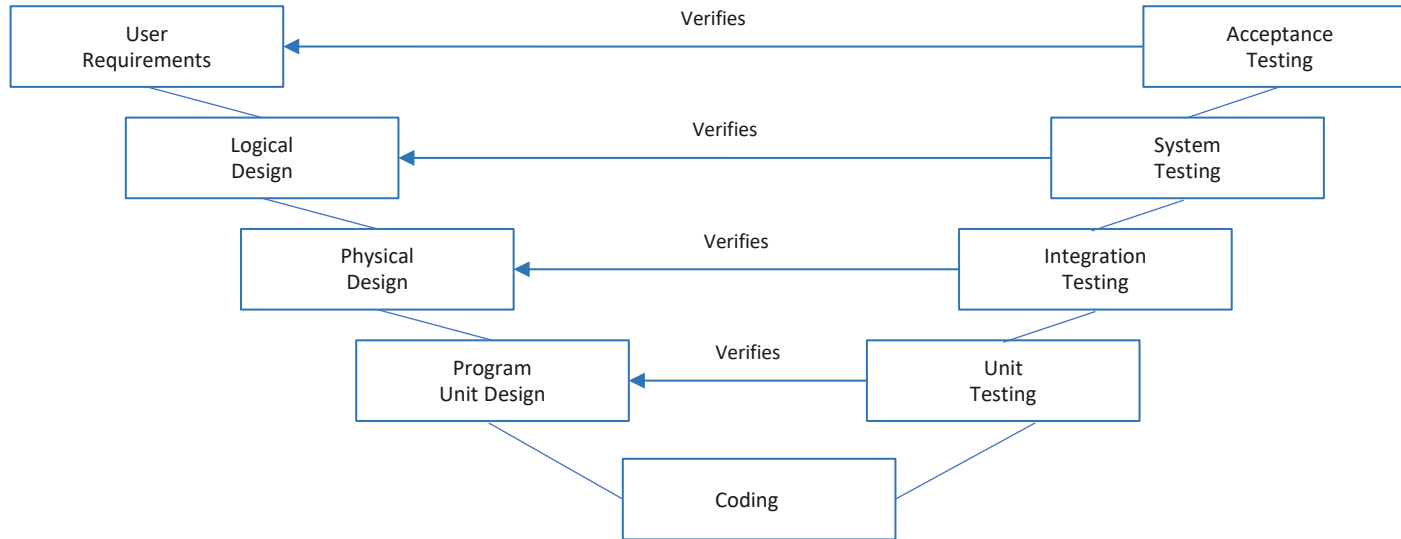


Figure 6.2 - Development phases versus testing types.

Test design runs downward: acceptance test criteria are established first in the test plan; as details are refined, the system, integration and unit testing requirements are established. Test execution runs upward: it starts with unit testing, then integration, then system, and finally the acceptance test.

PDCA Applied to Software Testing

PDCA Step	What It Means for Testing	Key Deliverables and Elements
PLAN	Starts with a definition of the test objectives - what is to be accomplished as a result of testing. Objectives ensure all responsible individuals contribute to the definition of test criteria.	Software test plan (major deliverable). An ongoing document: as the system changes, so does the plan. Becomes part of system maintenance documentation after delivery.
DO	Describes how to design and execute the tests included in the test plan. The more definitive the test plan, the easier the test design.	Test cases, test procedures and scripts, expected results, function/test case matrix, test logs. Selecting test tools; defining resource requirements, test setup conditions and environment.
CHECK	Evaluation of how the testing process is progressing. In God we trust, all others must use data. Base decisions on accurate and timely data.	Testing metrics: number and types of defects, workload effort, schedule status. Test reports - summary and interim, written at end of testing and at key checkpoints.
ACT	Devising measures for appropriate actions relating to work not performed according to the plan, or results not anticipated in the plan. This analysis is fed back to the plan.	Updating the test suites, test cases and test scripts; reevaluating the people, process and technology dimensions of testing.

The Testing Bible: The Software Test Plan

Definition

A test plan is a document describing the approach to be taken for intended testing activities. It serves as a service-level agreement between the quality assurance testing function and other interested parties, such as development.

What a test plan defines

- Test objectives and scope
- Strategy and approach
- Test procedures and test environment
- Test completion criteria
- Test cases; items to be tested
- Tests to be performed; test schedules
- Personnel requirements; reporting procedures
- Assumptions, risks, contingency planning

What makes a good test plan

- Simple, complete, current, and accessible by the appropriate individuals for feedback and approval
- Flows logically and minimises redundant testing
- Demonstrates full functional coverage
- Provides workable procedures for monitoring, tracking and reporting test status
- Contains a clear definition of the roles and responsibilities of the parties involved
- Has target delivery dates and clearly documents the test results

Two Approaches to Building a Test Plan

Approach 1 - Master Test Plan		Approach 2 - One Test Plan
Concept	An overview of each detailed test plan - that is, a test plan of test plans.	All the test types are included in one single test plan.
Structure	A detailed test plan verifies a particular phase in the waterfall development life cycle.	Often called the acceptance/system test plan, but covers unit, integration, system and acceptance testing.
Examples of detailed plans	Unit, integration, system and acceptance test plans. Others include application enhancements, regression testing, and package installation.	A single plan with all the planning considerations required to complete the tests.
Characteristics	Unit test plans are code oriented and very detailed but short, because of their limited scope. System or acceptance test plans focus on the functional or black-box view of the entire system, not just a software unit.	This is the approach used throughout Section 2 to describe how a test plan is built during the waterfall life-cycle development methodology.

The Test Case Form

Chapter 6 - Figure 6.3 - A major component of the Test Procedure section

Date: _____	Tested by: _____
System: _____	Environment: _____
Objective: _____	Test ID _____ Reg. ID _____
Function: _____	Screen: _____
Version: _____	Test Type: _____
(Unit, Integ., System, Accept.)	
Condition to test:	

Data/steps to perform	

Expected results:	

Actual results: Passed ____ Failed ____	

Figure 6.3 - Test case form.

A test case defines the step-by-step process whereby a test is executed. It includes the objectives and conditions of the test, the steps needed to set the test up, the data inputs, the expected results, and the actual results.

Major Steps in Developing a Test Plan

A test plan is a living document: as the application changes, the test plan should change

Step 1

Define the Test Objectives

Establish what is to be accomplished. Determines the specific tests to be performed, test expectations, critical success factors, constraints, scope, expected end products, and the final system summary report. The test objectives are reviewed and approval obtained.

Step 2

Develop the Test Approach

Outline how each test will be performed: testing techniques, test entry criteria, test exit criteria, procedures to coordinate with development, test management approach such as defect reporting and tracking, test progress tracking, status reporting, test resources and skills, risks, and definition of the test basis.

Step 3

Define the Test Environment

Examine the physical test facilities; define the hardware, software and networks; determine which automated test tools and support tools are required; define the help desk support required; build special software required for the test effort.

Step 4

Develop the Test Specifications

Form the test team to write the test specifications; develop format standards; divide up the work tasks and work breakdown; assign team members to tasks; identify features to be tested; cross-reference specifications to the functional specifications; identify interdependencies and work flow.

Step 5

Schedule the Test

Develop a test schedule based on resource availability and the development schedule; compare the schedule with deadlines; balance resources and workload demands; define major checkpoints; develop contingency plans.

Step 6

Review and Approve the Test Plan

Schedule a review meeting with the major players; review the plan in detail to ensure it is complete and workable; obtain approval to proceed.

Table 6.1 - System/Acceptance Test Plan versus Phase

Part 1 of 4 . Section 1 Introduction . Section 2 Test approach and strategy (a-d)

Test Section	Requirements Phase	Logical Design	Physical Design	Program Unit Design	Coding Phase
1. Introduction					
a. System description	Start	Refine	Refine	Complete	
b. Objective	Start	Refine	Refine	Complete	
c. Assumptions	Start	Refine	Refine	Complete	
d. Risks	Start	Refine	Refine	Complete	
e. Contingencies	Start	Refine	Refine	Complete	
f. Constraints	Start	Refine	Refine	Complete	
g. Approval signatures	Start	Refine	Refine	Complete	
2. Test approach and strategy					
a. Scope of testing	Start	Refine	Refine	Complete	
b. Test approach	Start	Refine	Refine	Complete	
c. Types of tests	Start	Refine	Refine	Complete	
d. Logistics	Start	Refine	Refine	Complete	

Legend: Start = recommended start time, or first cut of a test activity . Refine = a refinement of a test activity started in a previous life-cycle phase . Complete = the life-cycle phase in which the test activity is completed

Table 6.1 - System/Acceptance Test Plan versus Phase

Part 2 of 4 . Section 2 (e-m) . Section 3 Test execution setup

Test Section	Requirements Phase	Logical Design	Physical Design	Program Unit Design	Coding Phase
e. Regression policy	Start	Refine	Refine	Complete	
f. Test facility		Start	Refine	Complete	
g. Test procedures		Start	Refine	Complete	
h. Test organization		Start	Refine	Complete	
i. Test libraries		Start	Refine	Complete	
j. Test tools		Start	Refine	Complete	
k. Version control		Start	Refine	Complete	
l. Configuration building		Start	Refine	Complete	
m. Change control		Start	Refine	Complete	
3. Test execution setup					
a. System test process		Start	Refine	Complete	
b. Facility		Start	Refine	Complete	
c. Resources		Start	Refine	Complete	
d. Tool plan		Start	Refine	Complete	
e. Test organization		Start	Refine	Complete	

Legend: Start = recommended start time, or first cut of a test activity . Refine = a refinement of a test activity started in a previous life-cycle phase . Complete = the life-cycle phase in which the test activity is completed

Table 6.1 - System/Acceptance Test Plan versus Phase

Part 3 of 4 . Section 4 Test specifications . Section 5 Test procedures

Test Section	Requirements Phase	Logical Design	Physical Design	Program Unit Design	Coding Phase
4. Test specifications					
a. Functional decomposition	Start	Refine	Refine	Complete	
b. Functions not to be tested	Start	Refine	Refine	Complete	
c. Unit test cases				Start	Complete
d. Integration test cases			Start	Complete	
e. System test cases		Start	Refine	Complete	
f. Acceptance test cases	Start	Refine	Refine	Complete	
5. Test procedures					
a. Test case, script, data development	Start	Refine	Refine	Refine	Complete
b. Test execution	Start	Refine	Refine	Refine	Complete
c. Correction	Start	Refine	Refine	Refine	Complete
d. Version control	Start	Refine	Refine	Refine	Complete
e. Maintaining test libraries	Start	Refine	Refine	Refine	Complete
f. Automated test tool usage	Start	Refine	Refine	Refine	Complete
g. Project management	Start	Refine	Refine	Refine	Complete
h. Monitoring and status reporting	Start	Refine	Refine	Refine	Complete

Legend: Start = recommended start time, or first cut of a test activity . Refine = a refinement of a test activity started in a previous life-cycle phase . Complete = the life-cycle phase in which the test activity is completed

Table 6.1 - System/Acceptance Test Plan versus Phase

Part 4 of 4 . Section 6 Test tools . Section 7 Personnel resources . Section 8 Test schedule

Test Section	Requirements Phase	Logical Design	Physical Design	Program Unit Design	Coding Phase
6. Test tools					
a. Tools to use		Start	Refine	Complete	
b. Installation and setup		Start	Refine	Complete	
c. Support and help		Start	Refine	Complete	
7. Personnel resources					
a. Required skills	Start	Refine	Refine	Complete	
b. Roles and responsibilities	Start	Refine	Refine	Complete	
c. Numbers and time required	Start	Refine	Refine	Complete	
d. Training needs	Start	Refine	Refine	Complete	
8. Test schedule					
a. Development of test plan		Start	Refine	Refine	Complete
b. Design of test cases		Start	Refine	Refine	Complete
c. Development of test cases		Start	Refine	Refine	Complete
d. Execution of test cases		Start	Refine	Refine	Complete
e. Reporting of problems		Start	Refine	Refine	Complete
f. Developing test summary report		Start	Refine	Refine	Complete
g. Documenting test summary report		Start	Refine	Refine	Complete

Legend: Start = recommended start time, or first cut of a test activity . Refine = a refinement of a test activity started in a previous life-cycle phase . Complete = the life-cycle phase in which the test activity is completed

Technical Reviews as a Continuous Improvement Process

Quality control is a key preventive component of quality assurance. Defect removal via technical reviews during the development life cycle is an example of a quality control technique. The purpose is to increase the efficiency of the development life cycle and provide a method to measure the quality of the products. Technical reviews reduce the amount of rework, testing, and quality escapes - that is, undetected defects.

Origin

Originally developed by Michael Fagan of IBM in the 1970s. Inspections have several aliases and are often referred to interchangeably as peer reviews, inspections, or structured walkthroughs.

Where They Are Performed

At each phase of the development life cycle, from user requirements through coding. In the coding phase, code walkthroughs are performed in which the developer walks through the code for the reviewer.

Why Reviews Beat Testing Alone

Research demonstrates that technical reviews can be a lot more productive than automated testing techniques in which the application is executed and tested. A technical review is a form of testing - manual testing - not involving program execution on the computer.

Structured walkthroughs and inspections remove defects earlier in the life cycle, thereby reducing defect-removal costs significantly. They can potentially result in a reduction of defect-removal costs of greater than TWO THIRDS when compared to dynamic software testing.

Motivation for Technical Reviews . Types of Reviews

Why review at all?

- It is impossible to test all software; exhaustive testing of code is impractical.
- Technology does not exist for testing a specification or high-level design.
- The idea of testing a software test plan is bewildering.
- Testing does not address quality issues or adherence to standards.

What a review delivers

- Increases the quality of the software product
- Reduces rework and ambiguous efforts; reduces testing
- Defines test parameters; is a repeatable and predictable process
- Increases the reliability of the delivered product; positive impact on schedule; reduces development costs
- Early error detection reduces rework at later stages, clarifies requirements and design, identifies interfaces, reduces failures and retests, identifies requirements testability, and helps identify missing or ambiguous requirements

Formal versus informal reviews

Informal Reviews

Occur spontaneously among peers. The reviewers do not necessarily have any responsibility and do not have to produce a review report.

Formal Reviews

Carefully planned meetings in which reviewers are held responsible for their participation, and a review report is generated that contains action items.

The Review Spectrum

Ranges from very informal peer reviews to extremely formal and structured inspections. The complexity of a review is usually correlated to the complexity of the project - as project complexity increases, the need for more formal reviews increases.

Structured Walkthroughs versus Inspections

Dimension	Structured Walkthrough	Inspection
Nature	A presentation review. A review participant, usually the developer of the software being reviewed, narrates a description of the software; the remainder of the group provides feedback throughout the presentation.	A formally defined process for verification of the software product throughout its development. All software deliverables are examined at defined phases.
Preparation	Advance preparation of the reviewers is not necessarily required.	Requires an agenda to guide the review preparation and the meeting itself. Has rigorous entry and exit requirements for the project work deliverables.
Character	Because of its informal structure, disorganized and uncontrolled reviews may result. May also be stressful if the developer is conducting the walkthrough.	Very systematic, controlled, and less stressful. Promotes the concept of egoless programming - developers need not become emotionally protective of the work produced.
Feedback	Feedback is limited: the bulk of it usually occurs only for the material actually presented.	One of the major decisions within an inspection is whether a software deliverable can proceed to the next development phase.
Process focus	Focused on the product being presented.	Inspections collect information to improve the development and review processes themselves - so an inspection is more of a quality assurance technique than a walkthrough.

Cost of delay: software quality is achieved in a product during the early stages, when the cost to remedy defects is 10 to 100 times less than it would be during testing or maintenance. It is therefore advantageous to find and correct defects as near to their point of origin as possible.

Table 6.2 - PDCA Process and Inspections

Phased inspections apply the PDCA quality model to every development phase

Inspection Step	Description	Plan	Do	Check	Act
1. Planning	Identify participants, get materials together, schedule the overview	✓			
2. Overview	Educate for the inspections	✓			
3. Preparation	Individual preparation for the inspections		✓		
4. Inspection	Actual inspection to identify defects		✓	✓	
5. Rework	Rework to correct any defects				✓
6. Follow-up	Follow up to ensure all defects are corrected				✓

Plan

Inspection planning and preparing an education overview. Designate specific products, acceptable criteria and meaningful metrics. Inspection materials must meet inspection entry criteria.

Do

Individual preparation and the inspection itself. Each review is assigned specific aspects: technical accuracy, standards and conventions, quality assurance, and readability.

Check

Identification and documentation of the defects uncovered. Solution hunting and discussion of design alternatives are discouraged - inspections are a review process, not a solution session.

Act

The rework and follow-up required to correct any defects. The author reworks all discovered defects; the team ensures corrective actions are effective and no secondary defects are introduced.

Phased Inspections as an Ascending Spiral

Chapter 6 - Figure 6.4

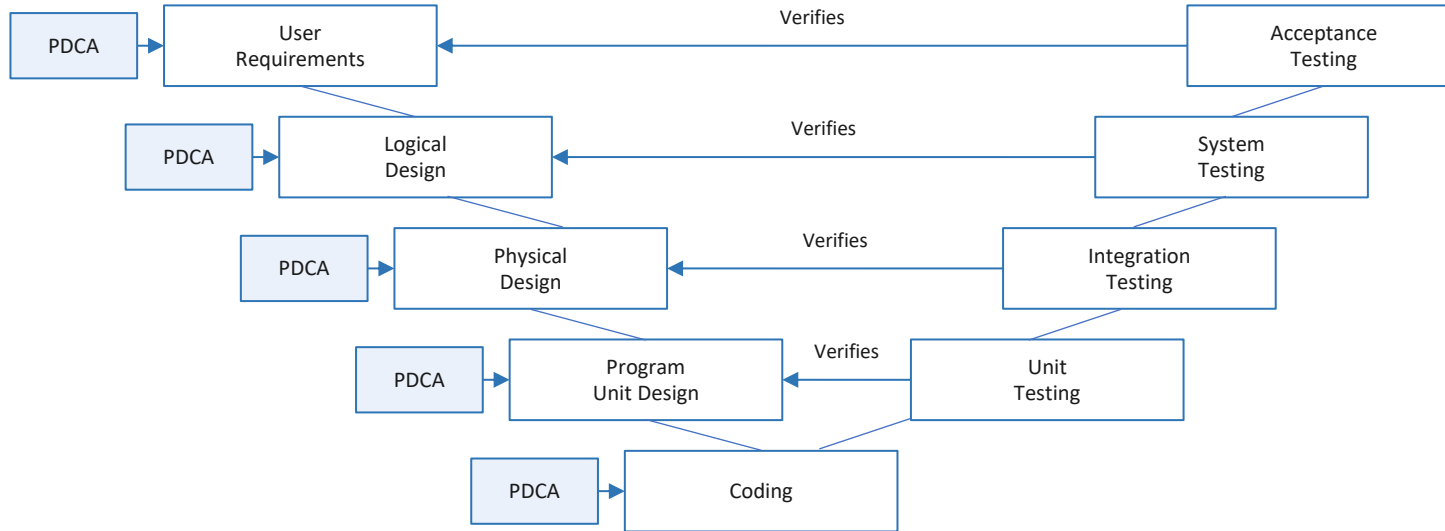


Figure 6.4 - Phased inspections as an ascending spiral.

By going around the PDCA cycle for each development phase using inspections, we verify and improve each phase deliverable at its origin and stop it dead in its tracks when defects are discovered. The next phase cannot start until the discovered defects are corrected. Repeated application of the PDCA results in an ascending spiral, facilitating quality improvement at each phase.

Participant Roles in a Review

Roles are functional - a participant may execute more than one role

Review Leader

Responsible for the review. Schedules the review, conducts an orderly review meeting, and prepares the review report. May also be responsible for ensuring action items are properly handled after the review.

Must possess both technical and interpersonal management characteristics: leadership ability, mediator skills, organizational talents. Must keep the review group focused at all times and prevent the meeting from becoming a problem-solving session. Material presented should not require more than two hours of preparation.

Recorder

Guarantees that all information necessary for an accurate review report is preserved.

Must understand complicated discussions and capture their essence in action items. Clearly a technical function and one that cannot be performed by a non-technical individual.

Reviewer

Objectively analyzes the software and is accountable for the review.

Must keep in mind that it is the SOFTWARE being reviewed and not the producer of the software - this cannot be overemphasized. The number of reviewers should be limited to six; if too many are involved, productivity will decrease.

Producer

May actually lead the meeting in an organized discussion of the software in a technical review.

A degree of preparation and planning is needed to present material at the proper level and pace. It is essential that the producer does not become defensive - the group leader should emphasize that the purpose is to uncover defects and produce the best product possible.

Steps for an Effective Review

Step 1 Plan for the Review Process

Planning happens at two levels. Organizational level: the number and types of reviews to be performed for the project, and allocation of project resources. Specific review level: selecting participants and defining their respective roles, scheduling the review, and developing a review agenda. Selecting review participants is a complex task normally performed by management with technical input. Care must be exercised to ensure that each aspect of the software under review can be addressed by at least some subset of the review team. To minimize stress and possible conflicts, it is important to discuss the role that a reviewer plays in the organization and the objectives of the review.

Step 2 Schedule the Review

A review should ideally take place soon after a producer has completed the software but before additional effort is expended on work dependent on it. The review leader must state the agenda based on a well-thought-out schedule. If all the inspection items have not been completed, another inspection should be scheduled. Allocating sufficient time is difficult; an effective technique is to estimate the time for each inspection item on a time line. Another scheduling problem is the duration of the review when it is too long. The deliverable to be reviewed should meet a certain set of entry requirements before the review is scheduled; exit requirements must also be defined.

Step 3 Develop the Review Agenda

A review agenda must be developed by the review leader and the producer prior to the review. Although review agendas are specific to any particular product and the objective of its review, generic agendas should be produced for related types of products. These agendas may take the form of checklists (see Appendix F, Checklists, for more details).

Step 4: Create a Review Report

The output of a review is a report - the format is not important, the contents are

The contents should address four perspectives.

Management Perspective

The review report serves as a summary of the review that highlights what was reviewed, who did the reviewing, and their assessment. Management needs an estimate of when all action items will be resolved, to successfully track the project.

User Perspective

The user may be interested in analyzing review reports for some of the same reasons as the manager. The user may also want to examine the quality of intermediate work products in an effort to monitor the development organization progress.

Developer Perspective

The critical information is contained in the action items. These may correspond to actual errors, possible problems, inconsistencies, or other considerations that the developer must address.

Quality Assurance Perspective

Twofold. QA must ensure that all action items in the review report are addressed. QA should also be concerned with analyzing the data on the review forms and classifying defects to improve the software development and review process.

Worked examples from the book: a large number of specification errors might suggest a lack of rigor or time in the requirements specifications phase of the project. . A large number of defects reported suggests that the software has not been adequately unit tested.